BÁO CÁO TIẾN ĐỘ TUẦN 3

Nghiên cứu kiến trúc ma trận 2D AES-128, cơ chế sinh khóa, PKCS#7 Padding, chế độ khối CBC và cài đặt mã hóa/giải mã File & ZIP đa năng

Học phần PBL3: Mạng thông tin máy tính

GVHD: TS. Trần Thị Minh Hạnh

SVTH: Nguyễn Thái Hoàng – 106230011

Lê Trương Uyển Nhi – 106230054

Tóm tắt nội dung

Tiếp nối kết quả nghiên cứu lõi thuật toán ở Tuần 2, nhóm tiến hành tái cấu trúc mã nguồn sang kiến trúc Ma trận 2D hướng đối tượng (lớp AES) theo chuẩn đặc tả Rijndael nhằm tăng tính mô-đun hóa. Nội dung trọng tâm của Tuần 3 bao gồm: hoàn thiện mô-đun giải mã đơn khối, xây dựng cơ chế sinh khóa CSPRNG và dẫn xuất khóa PBKDF2 từ mật khẩu, cài đặt cơ chế đệm dữ liệu PKCS#7 cùng chế độ hoạt động xích khối CBC. Trên cơ sở đó, nhóm mở rộng chương trình để mã hóa và giải mã luồng dữ liệu nhị phân cho các tệp tin đa định dạng và tệp nén ZIP, kiểm chứng việc khôi phục chính xác từng byte thông qua mã băm SHA-256. Quá trình biến đổi ma trận trạng thái State qua từng bước của Vòng 1 được tổng hợp trên cùng một bảng nhằm phục vụ việc đối chiếu giá trị trung gian.

Từ khóa: AES-128, Ma trận 2D, PKCS#7 Padding, CBC Mode, PBKDF2, Mã hóa File ZIP, SHA-256.

Bảng 1. Đối chiếu tiến độ kỹ thuật giữa Tuần 2 và Tuần 3

Tiêu chí kỹ thuật	Trạng thái Tuần 2	Nâng cấp trong Tuần 3
Kiến trúc mã nguồn	Mảng 1D (bytes) với chỉ số 4*c+r, hướng thủ tục.	Ma trận 2D 4x4 trực quan (class AES) theo chuẩn Rijndael.
Phạm vi xử lý	Chỉ mã hóa đúng 1 khối 16 byte cố định.	Xử lý dữ liệu độ dài bất kỳ nhờ cơ chế đệm PKCS#7.
Chế độ mã hóa	Mã hóa khối đơn (ECB block cipher).	Chế độ khối CBC (Cipher Block Chaining) với IV ngẫu nhiên.
Quản lý khóa	Nhập chuỗi hex 32 ký tự thủ công.	Tự sinh khóa ngẫu nhiên CSPRNG & dẫn xuất PBKDF2 từ mật khẩu.
Ứng dụng tệp tin	Chưa hỗ trợ mã hóa file.	Mã hóa & giải mã File / ZIP đa năng, khôi phục chính xác từng byte dữ liệu.

Bảng 2. Phân công nhiệm vụ thực hiện trong Tuần 3

Thành viên	Nhiệm vụ cụ thể trong Tuần 3	Đóng góp
Nguyễn Thái Hoàng (106230011)	Cài đặt class AES ma trận 2D, xây dựng PKCS#7 Padding, CBC Mode và mã hóa File ZIP.	50%
Lê Trương Uyển Nhi (106230054)	Cài đặt dẫn xuất khóa PBKDF2, bộ kiểm thử unit test 13 ca test và soạn thảo báo cáo.	50%

1. Giới thiệu và mục tiêu Tuần 3

Trong Tuần 2, nhóm đã nghiên cứu các phép biến đổi toán học cốt lõi của AES-128 theo chuẩn NIST FIPS 197 trên một khối 16 byte đơn lẻ. Tuy nhiên, để tiến tới việc ứng dụng mã hóa truyền tin trên giao thức TCP Socket ở Tuần 4, hệ thống cần giải quyết ba bài toán thực tế:

1. Dữ liệu thực tế có kích thước biến thiên: Cần cơ chế đệm (Padding) để chuẩn hóa kích thước dữ liệu về bội số của 16 byte.

2. Bảo mật khi mã hóa chuỗi khối liên tiếp: Cần chế độ hoạt động (Mode of Operation) như CBC nhằm triệt tiêu hiện tượng lộ mẫu lặp lại của dữ liệu.

3. Tái cấu trúc mã nguồn theo hướng đối tượng (OOP) và ma trận 2D: Đóng gói thuật toán thành lớp AES độc lập với biểu diễn ma trận 2D 4x4 (s[col][row]), giúp tách biệt tầng lõi mật mã với tầng ứng dụng xử lý tệp và truyền thông mạng ở các giai đoạn sau.

2. Kiến trúc giải thuật AES-128 ma trận 2D và bảng State Vòng 1

Khác với cách cài đặt tuần trước, State được biểu diễn dưới dạng danh sách 2 chiều gồm 4 cột, mỗi cột 4 phần tử: s = [[c0r0, c0r1, c0r2, c0r3], ...]. Kiến trúc biểu diễn ma trận 2D và các phép biến đổi lõi được nhóm tham khảo, đối chiếu từ mã nguồn mở của tác giả Lucas Boppre Niehues (boppreh/aes, giấy phép MIT) [3] cùng giáo trình The Design of Rijndael [2]. Phép chuyển đổi hai chiều giữa mảng byte phẳng và ma trận 2D được thực hiện qua hai hàm phụ trợ:

```
def bytes2matrix(text: bytes) -> list[list[int]]:
    return [list(text[i:i + 4]) for i in range(0, len(text), 4)]

def matrix2bytes(matrix: list[list[int]]) -> bytes:
    return bytes(b for col in matrix for b in col)
```

Các phép biến đổi cơ bản của AES-128 được hiện thực như sau:

• SubBytes / InvSubBytes: Duyệt qua từng ô s[i][j] và tra trực tiếp giá trị trong bảng S-box hoặc Inv S-box.

• ShiftRows / InvShiftRows: Dịch vòng các hàng của ma trận: Hàng 0 giữ nguyên; Hàng 1 dịch 1 vị trí; Hàng 2 dịch 2 vị trí; Hàng 3 dịch 3 vị trí.

• MixColumns / InvMixColumns: Áp dụng công thức đại số Rijndael rút gọn qua hàm xtime(a) = ((a << 1) ^ 0x1B) & 0xFF. Phép biến đổi InvMixColumns được tối ưu hóa theo sách The Design of Rijndael nhằm giảm thiểu số lượng phép nhân trên trường GF(2^8).

2.1 Biểu diễn trạng thái State qua các phép biến đổi của Vòng 1

Để đối chiếu sự thay đổi dữ liệu qua từng phép biến đổi trung gian của giải thuật, Bảng 3 biểu diễn chuỗi trạng thái State của Vòng 1 với vector kiểm thử chuẩn NIST FIPS 197

Phụ lục C.1 (Khóa K = 000102030405060708090a0b0c0d0e0f, Bản rõ P = 00112233445566778899aabbccddeeff):

1. Sau AddRoundKey (Round 0)	2. Sau SubBytes	3. Sau ShiftRows	4. Sau MixColumns	5. Kết quả sau AddRoundKey
00 40 80 c0 10 50 90 d0 20 60 a0 e0 30 70 b0 f0	63 09 cd ba ca 53 60 70 b7 d0 e0 e1 04 51 e7 8c	63 09 cd ba 53 60 70 ca e0 e1 b7 d0 8c 04 51 e7	5f 57 f7 1d 72 f5 be b9 64 bc 3b f9 15 92 29 1a	89 85 2d cb d8 5a 18 12 10 ce 43 8f e8 68 d8 e4

Chú thích cách đọc bảng: Để hiển thị thuận tiện trong trang văn bản, các ô trong Bảng 3 được trình bày theo thứ tự 4 hàng của ma trận (tương ứng với cấu trúc lưu trữ nội bộ s[col][row] theo thứ tự cột của chuẩn Rijndael). Cột 1 là trạng thái State ngay sau vòng khởi tạo AddRoundKey ban đầu (vòng 0), trước khi bước vào 4 phép biến đổi của Vòng 1.

3. Phương pháp thực hiện các mô-đun mở rộng

Trên nền tảng lõi mã hóa/giải mã đơn khối, nhóm tự thiết kế và cài đặt các mô-đun mở rộng gồm: cơ chế sinh khóa CSPRNG, dẫn xuất khóa PBKDF2, đệm dữ liệu PKCS#7, chế độ khối CBC và mô-đun mã hóa tệp tin nhị phân / ZIP.

3.1 Cơ chế tự động tạo khóa và dẫn xuất khóa (Key Derivation)

Nhóm triển khai hai phương thức quản lý khóa:

1. Sinh khóa ngẫu nhiên an toàn: Sử dụng os.urandom(16) từ bộ sinh số ngẫu nhiên an toàn mật mã của hệ điều hành (CSPRNG), đảm bảo các byte khóa độc lập và không thể đoán trước.

2. Dẫn xuất khóa từ mật khẩu (PBKDF2): Khi người dùng cung cấp mật khẩu văn bản (passphrase), hệ thống kết hợp với muối ngẫu nhiên (Salt 16 byte) và chạy qua thuật toán PBKDF2-HMAC-SHA256 (RFC 2898 / NIST SP 800-132) để tạo ra khóa 128-bit. Việc áp dụng hàm dẫn xuất có muối giúp làm chậm đáng kể các cuộc tấn công vét cạn (brute-force) và vô hiệu hóa bảng băm tính sẵn (rainbow tables). Số vòng lặp 100.000 là tham số học thuật thực nghiệm giúp cân bằng giữa thời gian dẫn xuất (<0.1 giây trên CPU phổ thông) và độ trễ chấp nhận được trong đồ án.

3.2 Cơ chế đệm dữ liệu chuẩn PKCS#7 (Padding & Unpadding)

Thuật toán mã hóa khối AES bắt buộc dữ liệu đầu vào phải là bội số của 16 byte. Nhóm cài đặt cơ chế đệm chuẩn PKCS#7 (RFC 5652). Gọi L là độ dài dữ liệu gốc tính theo byte. Số lượng byte cần đệm thêm P_len ($1 \leq P_len \leq 16$) được xác định theo công thức:

$$P_len = 16 - (L \bmod 16)$$

• Quy tắc thêm đệm: Chèn thêm đúng P_len byte, mỗi byte đều mang giá trị đúng bằng P_len. Trường hợp đặc biệt khi dữ liệu gốc đã chia hết cho 16 byte (L mod 16 = 0), hệ thống vẫn chèn thêm đủ một khối đệm 16 byte mang giá trị 0x10. Quy ước này giúp bên giải mã phân biệt chính xác dữ liệu gốc với byte đệm mà không làm cụt đuôi dữ liệu nhị phân.

• Quy tắc gỡ đệm: Đọc giá trị byte cuối cùng P_len, kiểm tra xem đủ P_len byte cuối có cùng giá trị P_len hay không. Nếu kiểm tra thất bại, hệ thống phát sinh ngoại lệ ValueError để cảnh báo dữ liệu bị giải mã sai hoặc bị tác động.

```
def pad(data: bytes, block_size: int = 16) -> bytes:
    padding_len = block_size - (len(data) % block_size)
    return data + bytes([padding_len] * padding_len)

def unpad(data: bytes, block_size: int = 16) -> bytes:
    if not data or len(data) % block_size != 0:
        raise ValueError('Dữ liệu không hợp lệ (không phải bội số của
block_size)')
    padding_len = data[-1]
    if padding_len < 1 or padding_len > block_size:
        raise ValueError('Padding PKCS#7 không hợp lệ: giá trị byte đệm
ngoài phạm vi 1-16')
    if data[-padding_len:] != bytes([padding_len] * padding_len):
        raise ValueError('Padding PKCS#7 bị hỏng hoặc sai giá trị kiểm
tra')
    return data[:-padding_len]
```

3.3 Chế độ hoạt động chuỗi khối CBC (Cipher Block Chaining)

Trong chế độ ECB, các khối bản rõ giống nhau sẽ tạo ra khối bản mã giống hệt nhau dưới cùng một khóa, dẫn đến nguy cơ lộ mẫu cấu trúc dữ liệu. Chế độ CBC (NIST SP 800-38A) khắc phục nhược điểm này bằng cách ghép xích các khối liên tiếp:

• Mã hóa CBC: Khối bản rõ đầu tiên được XOR với Vector khởi tạo (IV 16 byte) ngẫu nhiên: C0 = E_k(P0 ⊕ IV). Các khối tiếp theo được XOR với bản mã đứng trước: C_i = E_k(P_i ⊕ C_{i-1}).

• Giải mã CBC: Thực hiện ngược lại: P0 = D_k(C0) ⊕ IV và P_i = D_k(C_i) ⊕ C_{i-1}.

Đặc điểm an toàn: Mỗi lần mã hóa sử dụng một IV ngẫu nhiên mới (sinh từ os.urandom(16)), do đó cùng một nội dung tệp tin khi mã hóa nhiều lần sẽ cho ra các bản mã hoàn toàn khác nhau. Cần lưu ý rằng CBC chỉ đảm bảo tính bí mật (confidentiality), chưa cung cấp tính toàn vẹn (integrity) và không chống được việc kẻ tấn công cố tình sửa đổi các bit trong bản mã (malleability).

3.4 Cài đặt mô-đun mã hóa và giải mã File / ZIP đa năng

Tất cả các định dạng tệp tin trên hệ điều hành (.txt, .pdf, .docx, .zip) đều là luồng byte nhị phân. Mô-đun mã hóa tệp được tổ chức theo quy trình:

1. Đọc tệp gốc dưới dạng binary (rb) và tính giá trị băm SHA-256 để lưu làm mốc đối chiếu.

2. Áp dụng đệm PKCS#7 và mã hóa toàn bộ dữ liệu bằng chế độ AES-128-CBC.

3. Định dạng đóng gói tệp .enc: Nếu dùng khóa trực tiếp, lưu theo cấu trúc [16B IV] + [Ciphertext CBC]. Nếu dùng mật khẩu, hệ thống nhúng thêm tiền tố nhận diện và chuỗi muối: [8B Magic Header] + [16B Salt] + [16B IV] + [Ciphertext CBC].

4. Khi giải mã: Trích xuất IV (và Salt nếu có), giải mã CBC, gỡ đệm PKCS#7, ghi lại tệp gốc và đối chiếu mã băm SHA-256.

3.5 Hạn chế kỹ thuật hiện tại và định hướng phát triển

Qua quá trình thử nghiệm, nhóm ghi nhận hai hạn chế kỹ thuật cần được nâng cấp ở giai đoạn tiếp theo:

1. Nguy cơ về tính toàn vẹn và giải mã sai mật khẩu: Hiện tại mã băm SHA-256 chỉ được dùng đối chiếu độc lập, chưa nhúng làm mã xác thực mật mã (MAC) vào tệp .enc. Khi giải mã với mật khẩu sai, có xác suất xấp xỉ 1/256 (~0.39%) byte cuối cùng ngẫu nhiên rơi vào giá trị padding hợp lệ (ví dụ byte cuối bằng 0x01), khiến hàm unpad không báo lỗi và trả về dữ liệu rác. Để khắc phục triệt để, nhóm dự kiến bổ sung cơ chế Encrypt-then-MAC sử dụng HMAC-SHA256 trên toàn bộ gói tin (IV || Ciphertext) ở Tuần 4.

2. Quản lý bộ nhớ RAM: Chương trình hiện đọc toàn bộ tệp vào bộ nhớ, phù hợp cho các tệp vừa và nhỏ (<50 MB). Để phục vụ truyền dữ liệu qua giao thức mạng Socket ở Tuần 4, nhóm sẽ chuyển đổi sang cơ chế xử lý phân đoạn (Chunking Stream).

4. Thực nghiệm và kết quả kiểm thử

4.1 Kiểm thử tự động bằng thư viện unittest

Mã nguồn được kiểm thử bằng thư viện unittest trong tệp test_aes.py với 13 ca kiểm thử độc lập, bao gồm cả các vector chuẩn quốc tế của NIST và các trường hợp biên của tệp nhị phân:

Hạng mục kiểm thử	Nội dung & Dữ liệu kiểm tra	Kết quả thực nghiệm
Độ chuẩn xác FIPS 197	Vector C.1 và 4 vector chuẩn NIST AES_Core128.	5/5 vector PASS
Vector NIST SP 800-38A	Đối chiếu 4 khối CBC chuẩn Phụ lục F.2.1.	4/4 block PASS
Cơ chế đệm PKCS#7	Kiểm thử các độ dài biên: 0, 1, 2, 15, 16, 17, 31, 32, 100, 255 byte và từ chối đệm sai.	10/10 ca biên PASS

Hạng mục kiểm thử	Nội dung & Dữ liệu kiểm tra	Kết quả thực nghiệm
Dẫn xuất khóa PBKDF2	Tính tiền định với cùng muối và phân tách khóa từ mật khẩu.	PASS
Mã hóa chế độ CBC	Chuỗi văn bản UTF-8 và luồng nhị phân ngẫu nhiên đa khối.	PASS
Mã hóa File ZIP đa năng	Nén tài liệu thành file ZIP, mã hóa CBC, giải mã đối chiếu SHA-256.	PASS (Khớp từng byte)

4.2 Thực nghiệm mã hóa và giải mã tệp nén ZIP thực tế

Nhóm thực hiện kịch bản kiểm thử tự động (lệnh: python demo.py --demo-zip) với một tệp nén sample_archive.zip chứa 2 tệp tin (baocao_pbl3.txt và data/students.csv). Kết quả ghi nhận:

```
KẾT QUẢ THỰC NGHIỆM MÃ HÓA FILE ZIP (demo.py --demo-zip):
1. File ZIP gốc: sample_archive.zip (373 bytes)
   -> Mã băm SHA-256 gốc :
ed77c38073bd1126c19d0572161b4fd5a864b99a5fce78bc2cd12b07d3cce994
2. File sau mã hóa AES-128-CBC: sample_archive.zip.enc (400 bytes = 16B IV
+ 384B Ciphertext)
3. File sau khi giải mã: sample_archive_recovered.zip (373 bytes)
   -> Mã băm SHA-256 sau giải mã:
ed77c38073bd1126c19d0572161b4fd5a864b99a5fce78bc2cd12b07d3cce994
4. Đối chiếu mã băm toàn vẹn SHA-256: PASS (Trùng khớp 100% từng byte)
5. Mở và giải nén file ZIP: Đọc chính xác toàn bộ 2 file tài liệu bên trong
không có lỗi!
```

Giải thích kích thước dữ liệu: Tệp ZIP gốc có dung lượng 373 byte. Do 373 mod 16 = 5, cơ chế PKCS#7 đệm thêm 11 byte (0x0B) để tạo thành 384 byte (24 khối AES). Kết hợp với 16 byte IV ở đầu tệp, dung lượng tệp .enc sau mã hóa đạt đúng 400 byte.

5. Kết luận và kế hoạch Tuần 4

Trong Tuần 3, nhóm đã hoàn thành các nội dung mục tiêu đề ra:

• Tái cấu trúc thành công giải thuật AES-128 sang mô hình Ma trận 2D và Hướng đối tượng (lớp AES), tách biệt giữa tầng tính toán toán học và tầng ứng dụng tệp tin.

• Cài đặt cơ chế tạo khóa tự động (CSPRNG & PBKDF2), đệm dữ liệu PKCS#7 và chế độ hoạt động xích khối CBC.

• Thực hiện mã hóa và giải mã đối với tệp tin nhị phân và tệp ZIP, khôi phục chính xác cấu trúc và nội dung với đối chiếu mã băm SHA-256.

Kế hoạch Tuần 4: Ứng dụng mô-đun AES-CBC đã hoàn thiện để xây dựng chương trình Client - Server truyền nhận tệp tin qua giao thức mạng TCP Socket, đồng thời nghiên cứu tích hợp HMAC-SHA256 nhằm bảo vệ tính toàn vẹn thông điệp.

Tài liệu tham khảo

[1] National Institute of Standards and Technology (NIST), Advanced Encryption Standard (AES), FIPS PUB 197, 2001 (Updated 2023).

[2] Joan Daemen, Vincent Rijmen, The Design of Rijndael: AES - The Advanced Encryption Standard, Springer, 2002.

[3] Lucas Boppre Niehues (boppreh), Pure Python AES implementation, GitHub Repository: boppreh/aes, MIT License, 2020.

[4] RFC 5652, Cryptographic Message Syntax (CMS) - Section 6.3: Content-encryption process (PKCS#7 Padding).

[5] NIST Special Publication 800-38A, Recommendation for Block Cipher Modes of Operation: Methods and Techniques, 2001.

[6] NIST Special Publication 800-132, Recommendation for Password-Based Key Derivation: Part 1: Storage Applications, 2010.